Guia completo: Android Studio + MOCK do Ciclo

Este documento explica como abrir, executar, validar e integrar o aplicativo Ciclo ao MOCK. O projeto já vem configurado para iniciar com o mock embutido, sem servidor externo. O mock HTTP em Node.js é opcional e serve para demonstrar a troca de transporte sem alterar as telas.

1. Pré-requisitos

Instale o Flutter SDK compatível com o canal stable, o Android Studio, o Android SDK, um Android Emulator e os plugins Flutter e Dart do Android Studio. Para executar o mock HTTP, instale Node.js 18 ou superior.

Confirme no terminal:

```
flutter doctor
flutter --version
node --version
```

Corrija no Android Studio os itens marcados pelo `flutter doctor`, principalmente a licença do Android SDK e o dispositivo/emulador.

2. Abrir o projeto corretamente

1. Extraia o arquivo da entrega.
2. Abra o Android Studio.
3. Selecione **Open** e escolha a pasta `Ciclo`, não apenas a subpasta `android`.
4. Confirme que a pasta escolhida contém `pubspec.yaml`, `lib/`, `packages/` e `android/`.
5. Aguarde o carregamento do Flutter.
6. Abra o terminal integrado e execute:

```
flutter pub get
```

O comando instala também a dependência `http` usada pelo pacote local `packages/ciclo_core`.

3. Executar o app imediatamente

Inicie um emulador Android ou conecte um aparelho com Depuração USB ativa. Depois execute:

```
flutter devices
flutter run
```

No Android Studio, a mesma ação pode ser feita selecionando o dispositivo e pressionando **Run**.

A primeira tela já está configurada para usar o mock embutido. Use:

- E-mail: `lia@ciclo.app`
- Senha: `123456`

O login abre a tela Hoje e carrega os hábitos de exemplo. O app não depende de internet nessa configuração.

4. Como o mock embutido funciona

O fluxo de dados é separado em quatro camadas:

1. A tela chama um repositório, como `AuthRepository` ou `HabitRepository`.
2. O repositório monta o request do contrato.
3. `MockTransport` entrega o request ao `CicloMockServer`, que mantém os dados em memória.
4. O repositório transforma o JSON em modelos (`UserProfile`, `Habit` e `HabitPage`) para a interface.

A seleção fica em `lib/main.dart`:

```
runApp(CicloApp(services: AppServices.mock()));
```

A implementação do mock embutido está em `packages/ciclo_core/lib/src/mock_server.dart`. O contrato está em `docs/api.md`. As telas não precisam saber se os dados vieram da memória ou da rede.

5. Validar o fluxo funcional

Depois do login, valide nesta ordem:

1. Em **Hoje**, confira a lista e o progresso do dia.
2. Toque no círculo de um hábito para concluir e desfazer a conclusão.
3. Abra **Hábitos** e teste busca, filtros e criação.
4. Abra um hábito para testar detalhe, edição e exclusão.
5. Em **Perfil**, use **Simular falha do mock** e confirme a mensagem de erro; depois toque em tentar novamente.
6. Use **Restaurar exemplos** para repor os dados iniciais.
7. Abra a aba **Sistema** para conferir tokens, tipografia, botões, campos, cards, estados vazio e erro.

6. Executar os testes do app e do contrato

Com Flutter instalado:

```
flutter test
```

Para testar apenas o servidor HTTP em Node.js:

```
node --test mock/logic.test.mjs
```

Os testes verificam login, validação, autenticação por token, conclusão de hábito e criação de hábito.

7. Subir o MOCK HTTP opcional

Abra um terminal na raiz `Ciclo` e execute:

```
node mock/server.mjs
```

O servidor ficará disponível em:

```
http://localhost:4010/v1
```

Em outro terminal, confira a saúde do serviço:

```
curl http://localhost:4010/v1/health
```

A resposta esperada possui `"status": "ok"` e `"service": "ciclo-mock"`.

Deixe o terminal do Node aberto enquanto o app usar o transporte HTTP. O arquivo `mock/ciclo.http` contém chamadas de exemplo para o Android Studio/HTTP Client.

8. Apontar o app para o MOCK HTTP no emulador Android

O emulador Android não acessa o computador por `localhost`. Para ele, o computador é `10.0.2.2`.

1. Pare o app no Android Studio.
2. Abra `lib/main.dart`.
3. Substitua:

```
runApp(CicloApp(services: AppServices.mock()));
```

por:

```
runApp(CicloApp(
  services: AppServices.http("http://10.0.2.2:4010/v1"),
));
```

1. Salve o arquivo e execute novamente.
2. Entre com `lia@ciclo.app` e `123456`.
3. Abra **Perfil** e confira se o cartão Conexão mostra `HTTP http://10.0.2.2:4010/v1`.

O manifest Android já contém `INTERNET` e `android:usesCleartextTraffic="true"`, necessários para o servidor HTTP local durante o desenvolvimento.

9. Apontar para outros dispositivos

No navegador/desktop ou simulador iOS, use:

```
runApp(CicloApp(
  services: AppServices.http("http://localhost:4010/v1"),
));
```

Em um celular físico na mesma rede do computador, descubra o IP local do computador e use, por exemplo:

```
runApp(CicloApp(
  services: AppServices.http("http://192.168.0.10:4010/v1"),
));
```

Se o celular não conectar, verifique firewall, Wi-Fi e se o Node está escutando em `0.0.0.0`. O servidor deste projeto já faz isso.

10. Principais endpoints

Todas as rotas protegidas usam:

```
Authorization: Bearer {token}
Accept: application/json
Content-Type: application/json
```

Método	Rota	Uso
GET	`/health`	Verificar servidor; não exige token
POST	`/auth/login`	Autenticar e receber token
POST	`/auth/logout`	Encerrar sessão
GET	`/me`	Consultar usuário
GET	`/habits`	Listar hábitos
POST	`/habits`	Criar hábito
PATCH	`/habits/{id}`	Editar hábito
PATCH	`/habits/{id}/completion`	Marcar feito/pendente
DELETE	`/habits/{id}`	Excluir hábito
POST	`/dev/reset`	Restaurar exemplos do mock

11. Erros esperados e diagnóstico

- `422 VALIDATION_ERROR`: campos ausentes ou inválidos; a resposta pode trazer `error.fields`.
- `401 INVALID_CREDENTIALS`: e-mail conhecido com senha incorreta.
- `401 UNAUTHORIZED`: token ausente ou inválido.
- `404 NOT_FOUND`: rota ou hábito inexistente.
- `503 UPSTREAM_UNAVAILABLE`: falha simulada no mock embutido.

Se a lista não carregar no modo HTTP, confira primeiro se o Node está rodando, se a URL usa `10.0.2.2` no emulador e se `/health` responde no computador. Depois faça **Hot Restart** ou reinicie o app.

12. Checklist final para apresentação

- [ ] Android Studio abre a pasta `Ciclo` e encontra `pubspec.yaml`.
- [ ] `flutter pub get` termina sem erro.
- [ ] Um dispositivo aparece em `flutter devices`.
- [ ] O app inicia com `AppServices.mock()` sem servidor externo.
- [ ] Login da conta demo funciona.
- [ ] É possível concluir, criar, editar e apagar hábitos.
- [ ] Aba Sistema exibe os componentes.
- [ ] `node --test mock/logic.test.mjs` passa.
- [ ] O mock HTTP responde em `/v1/health`.
- [ ] `AppServices.http("http://10.0.2.2:4010/v1")` funciona no emulador.
- [ ] Perfil exibe o modo de conexão correto.

13. Arquivos que normalmente serão alterados

- `lib/main.dart`: escolha entre mock embutido e HTTP.
- `lib/app/app_services.dart`: composição dos repositórios e transporte.
- `packages/ciclo_core/lib/src/transport.dart`: transporte em memória ou HTTP.
- `packages/ciclo_core/lib/src/mock_server.dart`: regras do mock embutido.
- `mock/logic.mjs`: regras do mock HTTP.
- `docs/api.md`: contrato compartilhado.

Ao alterar um campo do JSON, atualize o contrato, o mock embutido, o mock HTTP e os testes juntos.